

Piano Tiles Project Report

By Muhammad Wasay Rafaqat & Shehryar Asif Lone

Table of Contents

1	Introduction
2	Literature Review
3	Block Diagram of Final Implementation
4	Design & Analysis
5	Components Used
6	Performance Analysis
7	Societal Relevance
8	Problems Faced During the Project and the Lab as a Whole
9	Suggestions for Improvement in Project and Lab Sessions as a Whole

Introduction

Digital logic circuits form the foundation of modern computing systems, interactive hardware, and real-time control applications. Even simple digital systems can demonstrate important ideas such as clocked behavior, sequential memory, combinational decision-making, state storage, and user interaction. This project applies these concepts by implementing a simplified version of the popular game **Piano Tiles** using only discrete digital logic components.

The objective of the project is to design and build a hardware-based, 4-column Piano Tiles arcade game without using any microcontroller, FPGA, or software-controlled processor. Instead of programming the game behavior, the system implements the game mechanics directly through 74-series logic ICs, 555 timer circuits, shift registers, counters, decoders, multiplexers, gates, latches, LEDs, pushbuttons, and a 7-segment display.

In the proposed system, falling tiles are represented using LEDs arranged in a 4-column display grid. Each column corresponds to one player input button. A master clock controls the downward movement of the tiles, while a separate high-speed counter and decoder circuit generates pseudo-random tile placement. When a tile reaches the bottom row, the player must press the corresponding button. If the button press matches the tile position, the system registers a successful hit and increments the score. If the player presses the wrong button or misses a tile, the system enters a Game Over state.

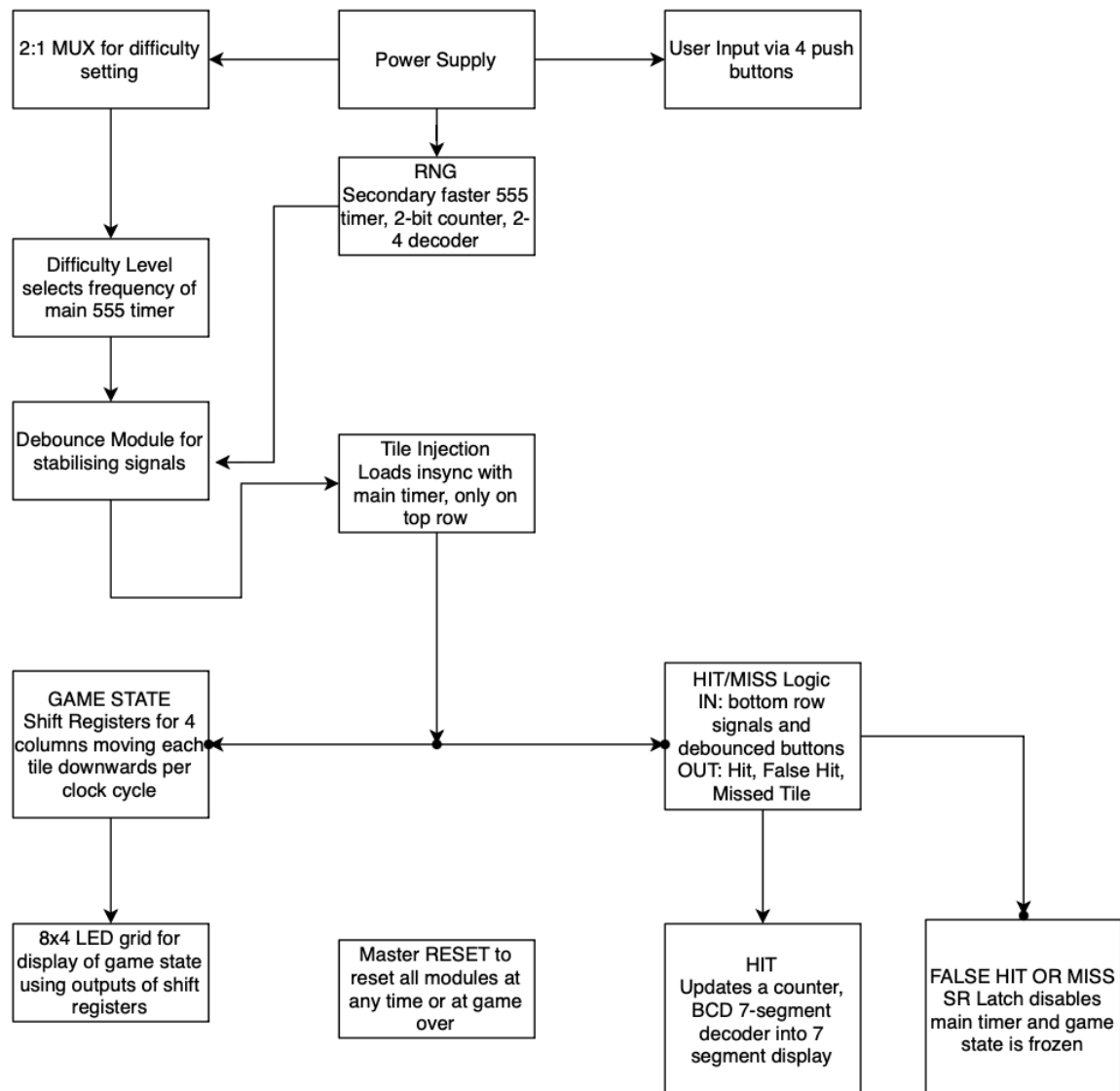
The system is divided into several functional blocks: master clock generation, pseudo-random tile generation, LED display movement, player input and debouncing, collision detection, score counting, difficulty selection, and Game Over state control. The design demonstrates both sequential logic, such as counters, shift registers, and latches, and combinational logic, such as AND/OR gates used for hit detection, false-press detection, and Game Over generation.

Lit Review

The following is the link for an implementation that uses fpgas however the core mechanics and flow are pretty much the same as our implementation.

<https://share.google/eRuy8MjJtFrp7X4lj>

Block Diagram



Design and Analysis

The design of the hardware Piano Tiles system was divided into separate modules so that each part could be simulated, tested, and debugged independently before integration. This

modular design approach reduced circuit complexity and made it easier to identify errors in timing, logic, or wiring.

Master Clock

The master clock acts as the game loop of the system. In a software game, the program repeatedly updates the screen and checks player input. In this hardware implementation, the same role is performed by a clock signal. Each clock pulse shifts the tile pattern one row downward through the LED grid.

An astable 555 timer is used to generate this periodic square wave. The frequency of this clock determines the game speed. A slower clock produces an easier game, while a faster clock increases difficulty by reducing the time available for the player to react.

The general timing equation for a 555 timer in astable mode is: $T = 0.693(R_a + 2R_b)C$

Where R_a and R_b are resistors and C is the capacitor. The frequency of the timer is the reciprocal of the timing equation. Different timing resistor and capacitor combinations can be used to generate multiple clock speeds. Which can then be put into a MUX to select a clock that is slow (easy difficulty) or fast (hard difficulty).

Tile Movement Using Shift Registers

The LED display is arranged as four columns and eight rows. Each column is controlled by one 8-bit Serial-In/Parallel-Out shift register. The parallel outputs of each shift register drive the LEDs in that column.

A tile is represented by a binary 1 stored inside the shift register. Empty spaces are represented by binary 0. When the master clock pulses, every bit moves to the next output stage. This creates the visual effect of a tile falling downward.

For example, if a tile enters at the top of a column, the bit pattern moves as follows:

Clock 1: $Q_0 = 1$

Clock 2: $Q_1 = 1$

Clock 3: $Q_2 = 1$

...

Clock 8: Q7 = 1

The second to bottom output, Q6, acts as the hit-zone signal for that column. The player must press the corresponding button when Q6 is HIGH.

This is a strong design choice because shift registers naturally implement motion without requiring a microcontroller, display memory, or complex addressing logic. Moreover, choosing the second last output as the hit-zone implicitly helps us choose the last output Q7 as the miss signal for that column. When the player did not hit and Q7 is high, we know the player has missed. This will also be explained later.

RNG Tile Generation

The system needs to choose which of the four columns receives the next tile. Since software randomness is not available, the design uses a hardware pseudo-random method based on asynchronous sampling.

A separate high-speed 555 timer drives a 74HC90 decade counter. The counter changes state much faster than the master game clock. Only the lower two bits of the counter are used, because two bits are enough to represent four columns:

00 → Column 1

01 → Column 2

10 → Column 3

11 → Column 4

These two counter bits are sent to a 74LS139 2-to-4 decoder. The decoder activates one of four output lines, selecting the column where the next tile will spawn. Because the RNG counter runs much faster than the game clock, the exact value captured by the slower tile-update process is difficult for the player to predict. This produces a practical pseudo-random effect suitable for the project.

Player Input and Debouncing

The player uses four pushbuttons, one for each column. Mechanical pushbuttons do not produce a perfect transition from LOW to HIGH. Instead, they often bounce, causing rapid unwanted pulses. If this bounce reaches the game logic, one button press may be interpreted as multiple presses, causing false hits or false Game Over conditions.

To reduce this problem, each button input is debounced using an RC network and Schmitt-trigger inverter such as the 74LS14. Pull-up or pull-down resistors are also used so that each input has a defined logic state when the button is not pressed. This prevents floating inputs, which can cause unpredictable behavior in simulation and hardware.

Collision and Detection Logic

Collision detection determines whether the player pressed the correct button at the correct time. For each column, two signals are checked:

B = Button signal

$L6$ = Q6 LED signal / hit-zone signal

A successful hit occurs when the button is pressed and a tile is present in the hit zone:

$$\text{Hit} = B \cdot L6$$

A false press occurs when the player presses the button while no tile is present:

$$\text{False Press} = B \cdot L6'$$

A missed tile occurs when a tile reaches the bottom row and is not hit before leaving the valid zone:

$$\text{FP} = \text{False Press}$$

$L7$ = Q7 LED signal / miss zone signal

$$\text{Miss} = L7 \cdot \text{FP}$$

Score Counting

The Game Over state is triggered by either of two failure conditions:

Game Over = FP + Miss

The failure signals from all columns are ORed together and connected to the Set input of an SR latch. The SR latch is built using NOR gates, such as the 74LS02. Once set, the latch holds the Game Over state even if the original failure signal disappears.

The output of the latch is used to freeze or disable the game clock, stopping tile movement. The latch remains active until the master reset button is pressed. This reset clears the Game Over state and prepares the system for a new game.

Modular Build Strategy

The system was designed and tested in stages. First, a single shift-register column was simulated to verify that LEDs could move downward on each clock pulse. Next, one button and one AND gate were added to test collision detection. After that, the high-speed counter and decoder were added to automate tile spawning. Once the single-column behavior was verified, the design was expanded to four columns. Finally, the score counter, difficulty multiplexer, and Game Over latch were integrated.

This modular strategy made the project more manageable because errors could be isolated within individual sections before combining the full circuit.

List of Components

2x NE555 Timers

1x 74HC90

1x 74LS193

1x 74LS157

4x 74HC595

1x 74LS139

2x 74LS14

3x 74LS32

3x 74LS08

1x 74LS02

33x Blue LEDs

1x 7 Segment Display Module (designed by lab)

1x D-Clock Module (designed by lab)

1x Debounce Switch Module (designed by lab)

1x Debounce Module (designed by lab)

5x Tactile Pushbuttons

7x 1k Ω Resistors

1x 2k Ω Resistor

1x 10k Ω Resistor

21x 0.1 μ F Capacitors

Software: Proteus 8 Professional, draw.io

Performance Analysis

The performance of the Piano Tiles hardware system was evaluated by testing each functional module separately and then verifying the integrated system. The main performance goals were correct tile movement, correct pseudo-random tile generation, accurate player input detection, proper score counting, reliable Game Over triggering, and successful reset behavior.

The link here showcases our core functionalities working as expected:

https://drive.google.com/file/d/1UiXsNTZcKWK1GZnmU5qCasYD6aBSRBtN/view?usp=share_link

The simulated circuit behaved close to the theoretical design because Proteus provides ideal digital transitions and stable component models. The hardware circuit may show small deviations due to practical effects.

Most notably, we saw noise creating imperfect square waves from the 555 timers that needed to be run through a debouncing circuit. The actual 555 timer frequency differed from the calculated value because resistor and capacitor combinations were not ideal (we could only select from the options available in the lab). Moreover, the frequency presented itself in a range which produced many composite numbers. When a frequency is to be divided into 4 different columns via a decoder and does not present itself to be a prime number, only some of the columns will sample. To counter this, we used a signal from the lab's D-clock module to sample column 2, while the other 3 columns were sampling from our RNG system.

Societal Relevance

Since this is an implementation of a video game, this project's impact is short-lived and limited to its implementation where we learnt several core principles of computations in hardware that can lead to further implementation of larger digital hardware systems or embedded systems.

Problems Faced

One of the biggest problems we faced was getting the 555 timers to settle on a specific frequency. As discussed above the frequency is varied by changing the capacitance or resistance applied to the IC. However the output of the IC not only had a lot of noise but it is also near impossible to set it to a single specific frequency also due to the noise. This causes difficulties leading to frequencies that either fail to sample any leds or that constantly sample always. To fix this we used a simple debouncing circuit to smooth the output, however this did not fix the fact that we were able to only find one frequency that properly randomly sampled leds in all columns (Further elaborated in performance analysis). Another issue we faced was a lack of breadboards which led to a messy implementation. We were able to complete the implementation on two breadboards, however the buttons were not positioned ergonomically enough for a single person to be able to press all four at the same time easily.

Suggestions

With more time available, we would have liked to make the implementation a bit more organised and shift the buttons to a smaller breadboard so it could be handheld and easier to play. Using solid core wires instead of the jumpers that crossed breadboards could help solve the same issue.